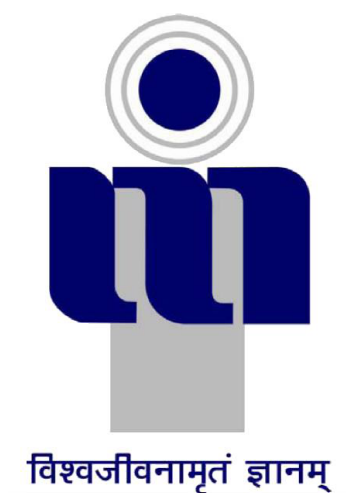


Summer Colloquium B.Tech Project

Development of High-Throughput Storage Engine

A report submitted in partial fulfillment of the requirements for the degree of B.Tech
in Mathematics and Scientific Computing



by

Avadh Khandelwal

2023BMS-007

**ATAL BIHARI VAJPAYEE INDIAN INSTITUTE OF
INFORMATION TECHNOLOGY AND MANAGEMENT**

Gwalior – 474015

2026

CANDIDATE'S DECLARATION

I, Avadh Khandelwal (2023BMS-007), hereby declare that the report titled

“Development of High-throughput Storage Engine”

has been prepared by me as part of the Summer Colloquium B.Tech Project for the degree of B.Tech in Mathematics and Scientific Computing at ABV-IIITM Gwalior. The work presented in this report was carried out by me from 15th May 2026 to 15th July 2026.

I further declare that this report is my original work and has not been submitted, in whole or in part, to any other institution for the award of any degree or academic qualification.

Date: _____ Signature of the Candidate: _____

This is to certify that the candidate's above declaration is true to the best of my knowledge.

Date: _____ Signature of the Project Supervisor: _____

ABSTRACT

This report details the architectural design and implementation of **ForgeLSM**, a **high-performance embedded C++ key-value storage-engine library**. It was built from the ground up to operate natively within application memory and implements a hybrid **WiscKey architecture** that physically decouples small keys from large values. Keys and metadata remain in a tightly compacted **LSM-Tree**, while values are appended to an efficient, zero-copy **Value Log (VLog)**. Furthermore, the engine provides **thread safety** and **lock-free concurrency** through a specialized **Skip List Memtable**. It provides **Single-Key ACID** guarantees for individual puts and deletes, together with **Snapshot Isolation** for reads through **Multi-Version Concurrency Control (MVCC)**; it provides no atomic multi-key operations.

Comprehensive benchmarking using custom and standard **YCSB workloads** demonstrates that ForgeLSM mitigates the severe I/O penalties of traditional LSM-Trees. By eliminating redundant value rewrites during compaction, the engine significantly reduces Write Amplification and has an **estimated 2.36× sustained Write Amplification Factor (WAF)** under continuous active garbage collection for standard 4 KB payloads. Integrated zero-I/O **Bloom Filters** are configured for a **98.9% theoretical rejection rate**; in missing-key benchmarks they empirically avoided **1,000,000 VLog reads** and more than **940,000 SSTable block searches**. Under strict hardware conditions requiring synchronous physical disk flushes, **Group Commit** enables the lock-free C++ architecture to scale synchronous write throughput. Independently, the **lock-free Skip List** and integrated Bloom Filters sustain more than **77,000 operations per second** in read-heavy workloads and more than **40,000 operations per second** for cold-disk random reads, while retaining microsecond-scale median read latencies.

Keywords: *ForgeLSM, embedded C++ database, key-value storage engine, WiscKey architecture, key-value separation, LSM-Tree, Value Log, lock-free concurrency, MVCC, Bloom Filters, Write Amplification.*

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to **Dr. Jeevaraj S.** for his invaluable guidance, thoughtful suggestions, and continuous encouragement throughout the development of this Summer Colloquium B.Tech project. His supervision and technical insights played an important role in shaping the design and implementation of ForgeLSM.

I am also thankful to the faculty members of the Mathematics and Scientific Computing programme at ABV-IIITM Gwalior for providing the academic environment and resources necessary to complete this work. I gratefully acknowledge my peers, friends, and family for their support and motivation during the course of the project.

Avadh Khandelwal

2023BMS-007

TABLE OF CONTENTS

ABSTRACT	ii
ACKNOWLEDGEMENTS	iii
LIST OF FIGURES	v
1 INTRODUCTION	1
1.1 Background and Motivation	1
1.2 Problem Statement	1
1.3 Project Scope	2
2 LITERATURE REVIEW	3
2.1 Log-Structured Merge Trees and Write Amplification	3
2.2 Leveraging Formats & Algorithms (WiscKey and Bloom Filters)	4
2.3 Resource Management & Concurrency (Skip Lists and Multithreading)	5
2.4 Enhancing Reliability (WAL and MVCC)	6
3 PROJECT OBJECTIVES AND DESIGN REQUIREMENTS	7
3.1 Overall Project Aim	7
3.2 Technical Objectives	7
3.3 Functional Requirements	8
3.4 Design Constraints and Non-Goals	8
3.5 Verification Criteria	9
4 METHODOLOGY AND IMPLEMENTATION	10
4.1 Engineering Methodology and System Architecture	10
4.2 Lock-Free Memtable and Monotonic Arena	11
4.3 WiscKey Separation, Value Log, and Zero-Copy I/O	12
4.4 LSM Compaction and Value-Log Garbage Collection	13
4.5 WAL, Group Commit, MVCC, and Crash Recovery	15

5	RESULTS AND BENCHMARKING	18
5.1	Experimental Setup and Evaluation Protocol	18
5.2	Impact of the WiscKey Architecture on Write Amplification	19
5.2.1	Theoretical Cost Model and Empirical Variance	19
5.3	Lock-Free Concurrency and Group Commit Scaling	21
5.4	True Cold-Disk Random Read Performance	21
5.5	Industry-Standard YCSB Workloads with Synchronous Durability . . .	22
5.5.1	Zero-I/O Bloom Filter Efficiency	23
5.6	Sustained Degradation and the Compaction Cliff	23
5.7	Dynamic Score-Based N-Level Cascading	24
5.8	Space Reclamation through Concurrent Garbage Collection	24
5.8.1	Real-World WAF under Active Garbage Collection	25
6	CONCLUSION	26
6.1	Limitations and Future Work	26
A	RAW BENCHMARK TRIALS	27
A.1	YCSB Workloads	27
A.2	Group Commit Scaling	28
A.3	Edge-Case Throughput	28

LIST OF FIGURES

2.1	Standard LSM-Tree architecture	4
2.2	WiscKey key-value separation	4
2.3	Bloom Filter membership test	5
2.4	Layered Skip List structure	6
4.1	High-level ForgeLSM architecture	11
4.2	Lock-free in-memory ingestion architecture	12
4.3	VLog pointer resolution	13
4.4	LSM compaction process	15
4.5	WAL leader-follower group commit	16
4.6	WAL recovery flow	17

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

Modern data-intensive applications require storage systems that can sustain high write rates while preserving predictable read latency and durable state. **Log-Structured Merge Trees (LSM-Trees)** address the cost of random in-place updates by buffering mutations in memory and flushing them to immutable **Sorted String Tables (SSTables)** on disk [1]. This design has become a foundation for widely deployed storage systems, including Google’s Bigtable [2].

The same architecture introduces an important trade-off. As SSTables accumulate, background compaction repeatedly merges and rewrites overlapping files. When keys are stored together with large values, these maintenance operations consume substantial I/O bandwidth, increase SSD wear, and can interfere with foreground requests. Key–value separation, as demonstrated by WiscKey, motivates retaining compact keys and metadata in the LSM-Tree while placing large values in a separate append-only log [4].

ForgeLSM was developed to study these storage-engine trade-offs through a complete embedded implementation. The project combines key–value separation with concurrent in-memory indexing, probabilistic filtering, snapshot-based reads, durable logging, compaction, and value-log reclamation.

1.2 Problem Statement

Traditional LSM-Tree designs repeatedly move both keys and values through the storage hierarchy during compaction. For workloads containing large values, this coupling produces severe **Write Amplification**: a comparatively small logical update may cause substantially more physical data to be written as the record cascades through multiple levels. The resulting background traffic competes with foreground operations for CPU

time and storage bandwidth.

The engineering problem addressed by ForgeLSM is therefore to design an embedded key-value storage engine that reduces large-value rewrite costs without sacrificing concurrent ingestion, efficient point lookups, crash-safe single-key durability, or consistent snapshot reads. The design must also manage the secondary consequences of key–value separation, particularly non-contiguous range access and the accumulation of obsolete records in the append-only Value Log.

1.3 Project Scope

ForgeLSM is a **C++20 embedded storage-engine library** that runs inside the host application’s process and memory space. Cross-platform operation was a design requirement from the beginning: compile-time branches abstract POSIX I/O on Unix-like systems and native Win32 APIs on Windows. The public interface supports **Put**, **Get**, **Delete**, range scans, and **PutBatch**. The **PutBatch** method is only a convenience wrapper over independent puts and provides neither crash atomicity nor atomic reader visibility across the batch.

The consistency scope is deliberately limited to **Single-Key ACID** guarantees for individual mutations and **Snapshot Isolation** for reads through Multi-Version Concurrency Control (MVCC). ForgeLSM provides no atomic multi-key operations or interactive transaction API. SQL parsing, networking and client–server protocols, replication, and distributed-cluster operation are outside the project scope; such facilities may be implemented by a higher-level database system using ForgeLSM as its embedded storage layer.

CHAPTER 2

LITERATURE REVIEW

2.1 Log-Structured Merge Trees and Write Amplification

The foundational architecture of modern high-performance databases relies on **Log-Structured Merge Trees (LSM-Trees)**, a concept introduced by O’Neil et al. [1] and successfully deployed in industry by Google’s Bigtable [2]. Traditional database engines commonly utilize **B-Trees**, which update data in place. Although efficient for point reads, in-place updates require random disk I/O and are therefore constrained by the physical latency of storage media.

LSM-Trees invert this paradigm by buffering incoming writes in memory using **Memtables** and subsequently flushing them sequentially to immutable disk files known as **Sorted String Tables (SSTables)**. This design allows LSM-Trees to achieve write speeds primarily bounded by the sequential-write bandwidth of the underlying storage device.

However, as highlighted by Dong et al. [3] in their analysis of RocksDB, the principal trade-off of LSM-Trees is the need for background **compactions**. To prevent read performance from degrading, background threads must continually merge and re-sort overlapping SSTables. This process causes severe **Write Amplification**: a single user-level write may trigger multiple internal rewrites of the same data, consuming substantial CPU and I/O resources, particularly for large payloads.

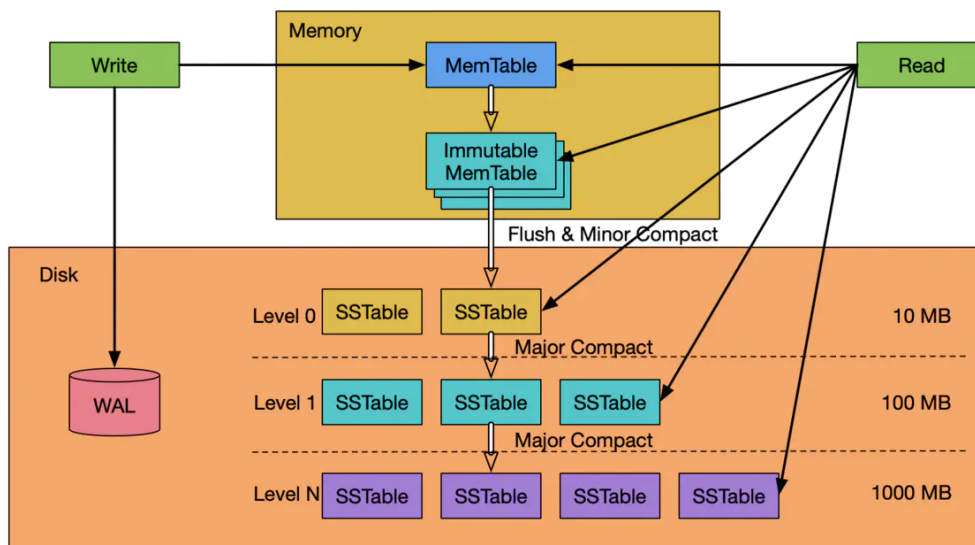


Figure 2.1: Standard architecture of a Log-Structured Merge Tree, showing the WAL, mutable and immutable Memtables, and multi-level SSTable hierarchy [1, 2].

2.2 Leveraging Formats & Algorithms (WiscKey and Bloom Filters)

To address the **Write Amplification** inherent in standard LSM-Trees, Lu et al. [4] proposed the **WiscKey architecture**. WiscKey introduces **key-value separation** by maintaining only keys and metadata within the LSM-Tree while appending the corresponding values sequentially to an uncompactd **Value Log (VLog)**. By decoupling large values from the compaction process, WiscKey reduces the theoretical Write Amplification from $O(N)$ towards $O(1)$, significantly preserving SSD endurance and sustaining write throughput.

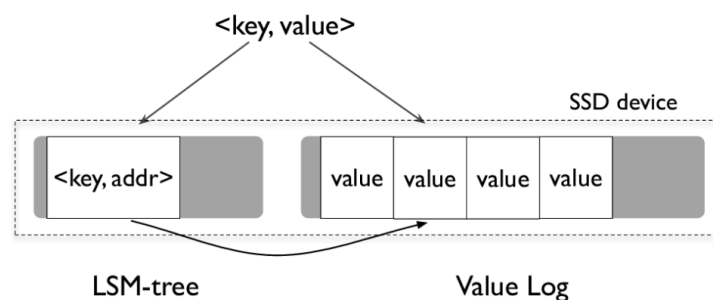


Figure 2.2: WiscKey key-value separation: the LSM-Tree stores compact key-address pairs, while the corresponding values are appended to a separate Value Log [4].

For point lookups, secondary-storage access remains a primary bottleneck. Burton H. Bloom [5] introduced **Bloom Filters**, a space-efficient probabilistic data structure used to test set membership. By passing a query through k independent hash functions into a bit array, a Bloom Filter can definitively determine whether a key is absent, avoiding unnecessary disk reads. Although false positives are possible, embedding a Bloom Filter within each SSTable block optimizes overall I/O by filtering out the majority of non-existent key queries before they reach the storage medium.

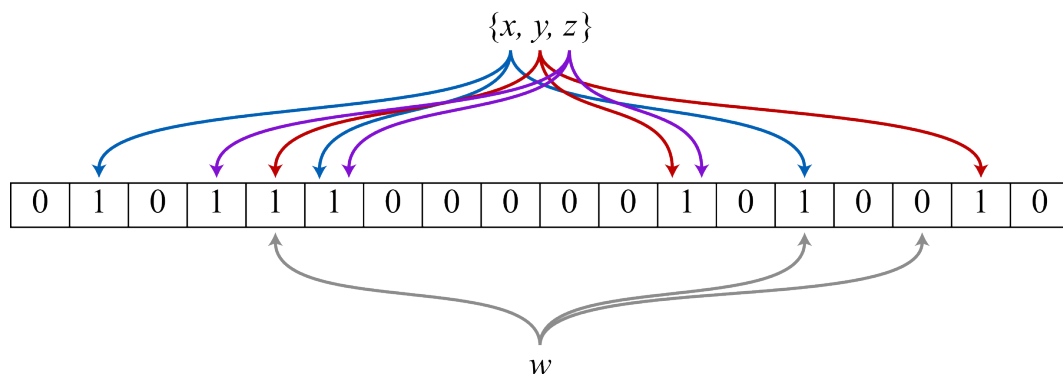


Figure 2.3: Probabilistic set-membership testing using multiple hash functions and a Bloom Filter bit array [5]. Illustration by David Eppstein, released into the public domain [6].

2.3 Resource Management & Concurrency (Skip Lists and Multithreading)

In high-throughput environments, **in-memory concurrency** is a critical performance factor. Traditional LSM-Tree Memtables often rely on balanced search trees, such as Red-Black Trees, protected by coarse-grained mutex locks. Under heavy multithreaded workloads, lock contention induces severe thread starvation and latency spikes.

Pugh [7] introduced the **Skip List**, a probabilistic alternative to balanced trees that uses randomized, layered linked lists to achieve expected $O(\log N)$ search, insertion, and deletion time. Crucially, Skip Lists can be implemented lock-free using atomic **Compare-And-Swap (CAS)** instructions. A **lock-free Skip List Memtable** allows multiple writer threads to mutate the structure concurrently without preempting reader threads, effectively eliminating mutex contention and scaling throughput with the available hardware threads.

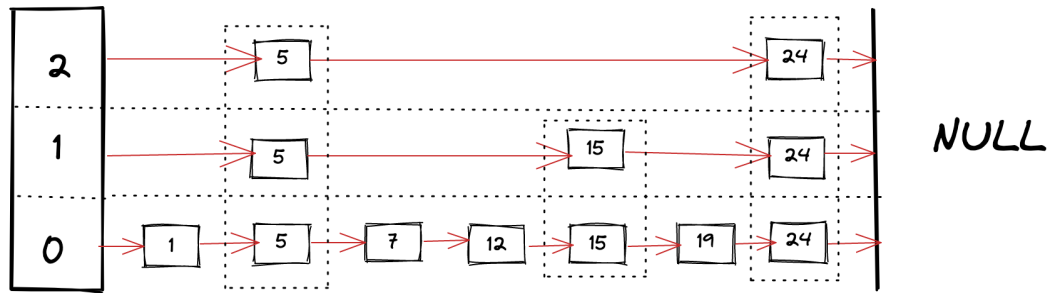


Figure 2.4: Layered structure of a Skip List supporting expected $O(\log N)$ traversal [7].

2.4 Enhancing Reliability (WAL and MVCC)

Database durability is typically ensured through a **Write-Ahead Log (WAL)** [8], an append-only file in which operations are durably recorded using `fsync` before being applied to in-memory data structures. To mitigate the I/O bottleneck caused by sequential physical syncs across multiple concurrent threads, systems employ **Group Commit** algorithms. By batching concurrent WAL append requests into a single physical disk flush, Group Commit amortizes the I/O cost and scales durability throughput.

To maintain transactional isolation without read locks, **Multi-Version Concurrency Control (MVCC)** [9] assigns monotonically increasing sequence numbers to each mutation. Rather than overwriting existing records, MVCC treats every update as a new version. Reader threads are assigned a snapshot sequence number at the start of an operation, allowing them to traverse the data as it existed at that exact moment. This guarantees that read operations remain consistently isolated from background compactions and active writer threads.

CHAPTER 3

PROJECT OBJECTIVES AND DESIGN REQUIREMENTS

3.1 Overall Project Aim

The aim of this project is to study, design, and implement **ForgeLSM**, a cross-platform embedded key-value storage engine written from scratch in **C++20**. The implementation provides a practical environment for examining how storage layout, concurrency control, probabilistic indexing, durability, and background maintenance interact within an LSM-based engine.

3.2 Technical Objectives

The project was guided by the following qualitative architectural objectives. These objectives describe the intended design behaviour and are kept separate from the measured benchmark outcomes reported later.

1. **Reduce Write Amplification through Key–Value Separation:** Implement a WiscKey-style layout in which keys and compact metadata remain in the LSM-Tree while large values are appended to a separate **Value Log (VLog)**. The objective is to reduce the amount of value data rewritten during compaction, particularly for large payloads.
2. **Support Highly Concurrent Ingestion:** Build the Memtable around a **lock-free Skip List** and monotonic Arena allocator so that foreground operations do not depend on a global Memtable mutex. Use **Group Commit** to coalesce the physical I/O of independent durable writes without implying multi-key atomicity.
3. **Mitigate Unnecessary Read I/O:** Integrate probabilistic **Bloom Filters** with

SSTables so missing-key lookups can reject irrelevant files or blocks before attempting expensive secondary-storage access.

4. **Provide Crash-Safe Single-Key Durability and Consistent Reads:** Use a **Write-Ahead Log (WAL)**, record checksums, and recovery logic to preserve individual mutations across process failure. Use **MVCC** sequence numbers to provide point-in-time, snapshot-isolated reads during concurrent writes and background maintenance.
5. **Maintain an Embedded, Cross-Platform Design:** Keep the engine independent of networking and query-language layers while abstracting operating-system-specific file mapping and synchronization behind POSIX and Win32 implementations.

3.3 Functional Requirements

ForgeLSM must expose the core operations expected from an embedded ordered key-value engine:

- **Put:** insert a new key or replace the current value of an existing key;
- **Get:** resolve the newest version visible to the reader's snapshot;
- **Delete:** record a tombstone that logically removes a key;
- **Range Scan:** traverse ordered keys across a specified interval; and
- **PutBatch:** submit multiple independent puts through a convenience API.

The PutBatch interface is not a transaction mechanism. Each enclosed mutation retains independent single-key durability and visibility semantics.

3.4 Design Constraints and Non-Goals

The system is constrained to operate as an in-process library under the C++20 language standard. It must account for different operating-system file and mapping interfaces

without changing its storage semantics. The project does not attempt to provide SQL parsing, remote client access, replication, distributed consensus, cluster partitioning, rollback-capable transactions, or atomic multi-key operations.

3.5 Verification Criteria

The design is considered successfully implemented when component and stress tests demonstrate that it can ingest and retrieve data concurrently, flush and compact SSTables through multiple levels, resolve values from the VLog, reclaim obsolete VLog records, reject missing-key reads through Bloom Filters, and recover valid single-key state from the WAL after controlled process termination. Quantitative performance and reliability observations are presented separately in the Results and Benchmarking chapter.

CHAPTER 4

METHODOLOGY AND IMPLEMENTATION

4.1 Engineering Methodology and System Architecture

ForgeLSM was developed using a bottom-up, component-oriented systems-engineering process. Each layer was implemented only after the invariants of the layer beneath it had been exercised through focused tests. Development progressed from the in-memory write path to persistent value storage, background maintenance, and finally durability and recovery. Stress testing and fault injection were used throughout the process to expose synchronization, file-layout, and recovery defects before system-level benchmarking.

The source code, benchmark and test suites, and original diagrams are publicly available in the ForgeLSM repository [12].

Figure 4.1 presents the high-level organization of the storage engine. Foreground writes pass through the WAL and active Memtable, large values are stored in the VLog, and immutable key–pointer entries are flushed into Bloom-filtered SSTables. Background workers perform compaction and VLog reclamation without changing the logical snapshot observed by active readers.

ForgeLSM is compiled under the **C++20** language standard and was designed for cross-platform operation from the outset. Compile-time branches using `#ifdef _WIN32` select the native Win32 implementation. On Windows, memory mapping is provided by

`CreateFileMappingA` and `MapViewOfFile`,

while durable synchronization uses `FlushFileBuffers`. Unix-like builds use the corresponding POSIX mapping, file, and synchronization calls. This abstraction keeps the higher-level storage algorithms independent of the host operating system.

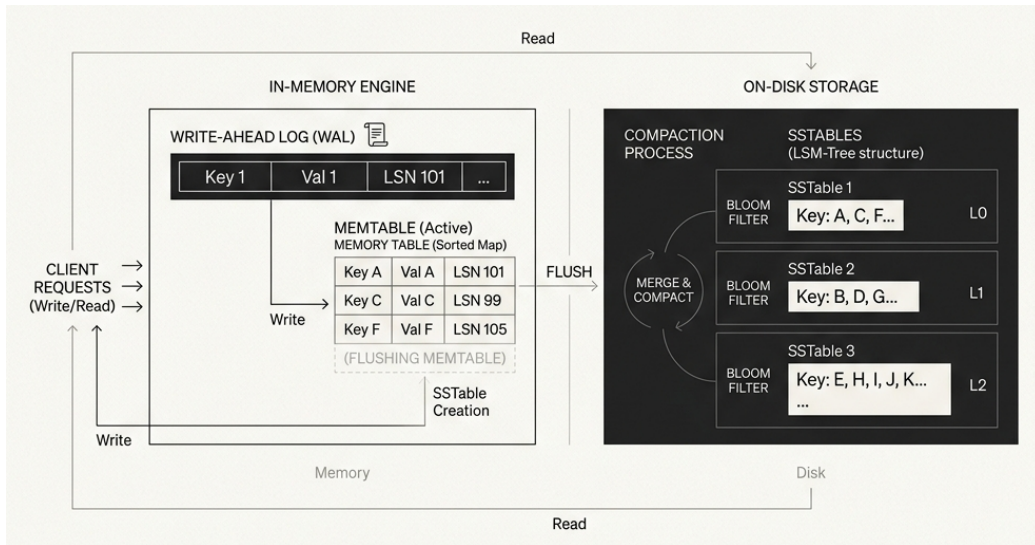


Figure 4.1: High-level ForgeLSM architecture showing the write-ahead log, active Memtable, Bloom-filtered SSTables, read path, and background compaction.

4.2 Lock-Free Memtable and Monotonic Arena

The initial phase addressed the memory-ingestion bottleneck. A naïve `std::map`-based approach introduces significant overhead because each operation requires $O(\log N)$ tree traversal, while mutex synchronization and non-contiguous general-purpose heap allocations can cause CPU cache-line thrashing and thread starvation.

To address this limitation, a custom lock-free `ConcurrentSkipList` was engineered as the primary Memtable. The SkipList maintains expected $O(\log N)$ search complexity through randomized, multi-level linked-list “express lanes” [7]. Concurrent updates are performed using atomic Compare-And-Swap (CAS) operations with explicit `memory_order_acquire` and `memory_order_release` semantics. Consequently, threads can continue to make progress without relying on a global mutex.

Memory allocation was also restructured to remove the allocator bottleneck from the lock-free data path. A custom monotonic Arena allocator pre-allocates memory in contiguous, multi-megabyte slabs. When a new SkipList node is required, the arena advances an offset through `std::atomic<size_t>::fetch_add` and enforces 8-byte alignment using

$$\text{aligned_ptr} = (\text{ptr} + 7) \& \sim 7. \quad (4.1)$$

This design places nodes close together in memory, improves spatial locality in the

CPU caches, and avoids repeated interaction with synchronization mechanisms inside the general-purpose malloc subsystem. The resulting ingestion path is illustrated in Figure 4.2.

To avoid the **ABA problem** and use-after-free defects in the lock-free data path, individual Skip List nodes are never freed during active ingestion. The monotonic Arena allocates nodes sequentially, and the entire Arena is released only after the Memtable becomes immutable, has been flushed to an SSTable, and all active reader threads holding snapshot references have safely detached.

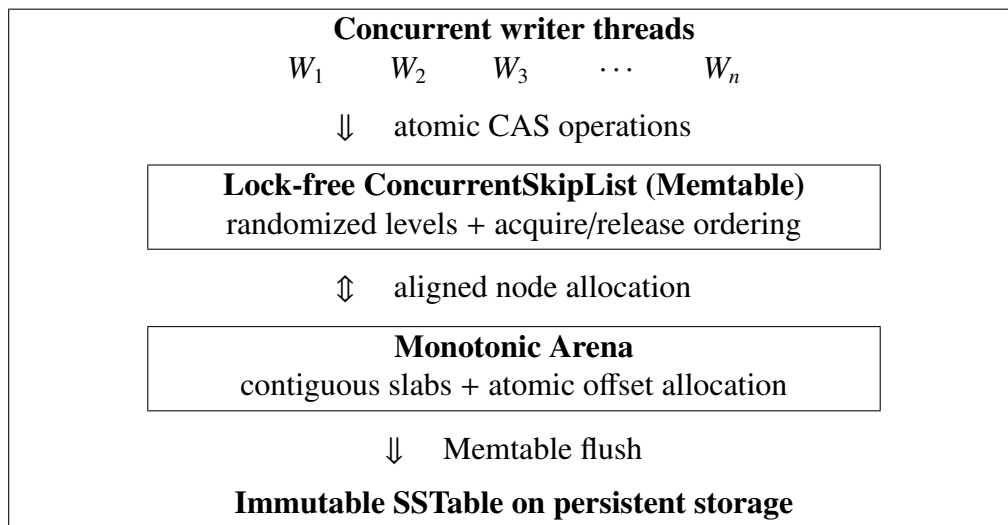


Figure 4.2: Lock-free in-memory ingestion architecture showing concurrent writers, atomic Skip List operations, monotonic Arena allocation, and the Memtable flush path.

4.3 WiscKey Separation, Value Log, and Zero-Copy I/O

After stabilizing the in-memory ingestion path, the persistent-storage design addressed the write amplification caused by traditional LSM-tree compaction. In conventional LevelDB- and RocksDB-style architectures, large values can be rewritten repeatedly as SSTables are merged through the storage hierarchy. ForgeLSM mitigates this behavior by adopting the WiscKey key–value separation paradigm [4].

The LSM-tree stores each user key together with a fixed 16-byte VLogPointer. Its three fields are

```
uint32_t file_id, uint64_t offset, and uint32_t length.
```

The corresponding payload is appended sequentially to an unstructured Value Log (VLog) using the record layout

$$[\text{key_size}][\text{value_size}][\text{key_bytes}][\text{value_bytes}]. \quad (4.2)$$

For point lookups, the VLog avoids an explicit `read()` system call for every value. Instead, POSIX `mmap()` maps the VLog into the process address space and permits the engine to resolve a `VLogPointer` through direct memory access. A resident page can be served from the operating system page cache with very low latency; when a page is not resident, the operating system transparently faults in the required block from the storage device. The lookup path is shown in Figure 4.3.

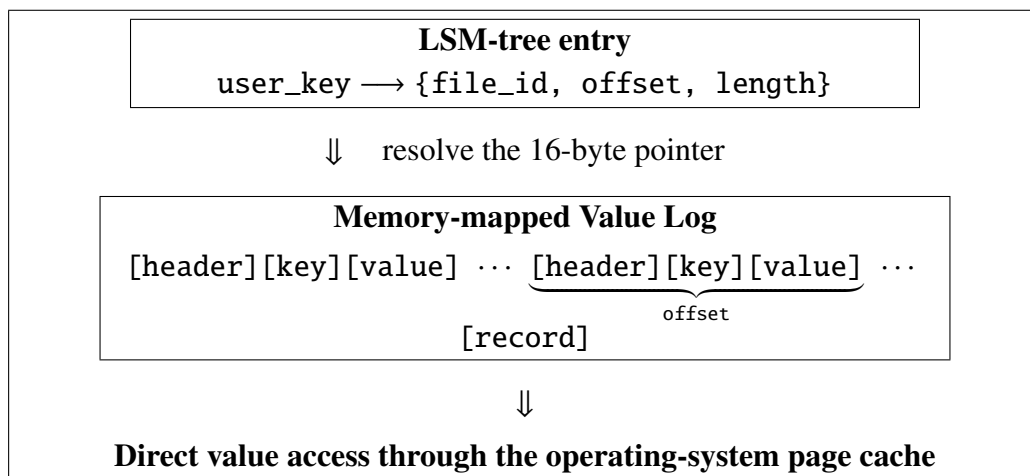


Figure 4.3: Resolution of a 16-byte `VLogPointer` within an `mmap`-backed VLog file, from the LSM-tree key–pointer entry to direct value access through the operating-system page cache.

4.4 LSM Compaction and Value-Log Garbage Collection

With keys and values structurally decoupled, secondary-storage management was designed to operate out of the foreground write path. Because the VLog is append-only, deleted or overwritten records leave dead physical bytes on the SSD. A garbage-collection (GC) protocol was therefore implemented to reclaim this space.

The GC thread sequentially scans VLog records and decodes their binary headers. For each embedded key, it queries the active LSM-tree and compares the returned

VLogPointer with the physical location currently being scanned. A record is considered *live* only when the active pointer matches that location; the record is then copied to a fresh VLog file. Records whose pointers no longer match are stale and are omitted. After all live records have been relocated safely, the obsolete file can be removed with `unlink()`, thereby reclaiming its SSD blocks.

To prevent a race in which a user overwrites a key while the garbage collector is relocating its older value, GC-initiated pointer updates are not installed through a separate maintenance path. They are routed through the primary lock-free write queue with the `is_gc` flag, ordering each relocation update against concurrent user writes. This shared queue provides atomic serialization for the single-key pointer replacement and prevents a stale GC relocation from bypassing the normal write-ordering mechanism.

Concurrently, the LSM-tree uses a K -way merge to compact overlapping SSTables through the physical levels,

$$L_0 \longrightarrow L_1 \longrightarrow \cdots \longrightarrow L_N. \quad (4.3)$$

Because large values remain in the VLog, compaction processes dense sequences containing only keys and 16-byte pointers. This reduction in the number of bytes rewritten mitigates the “compaction cliff” and helps maintain steadier write throughput as payload size increases [3]. Figure 4.4 illustrates the priority-queue-based K -way merge used to combine overlapping SSTables.

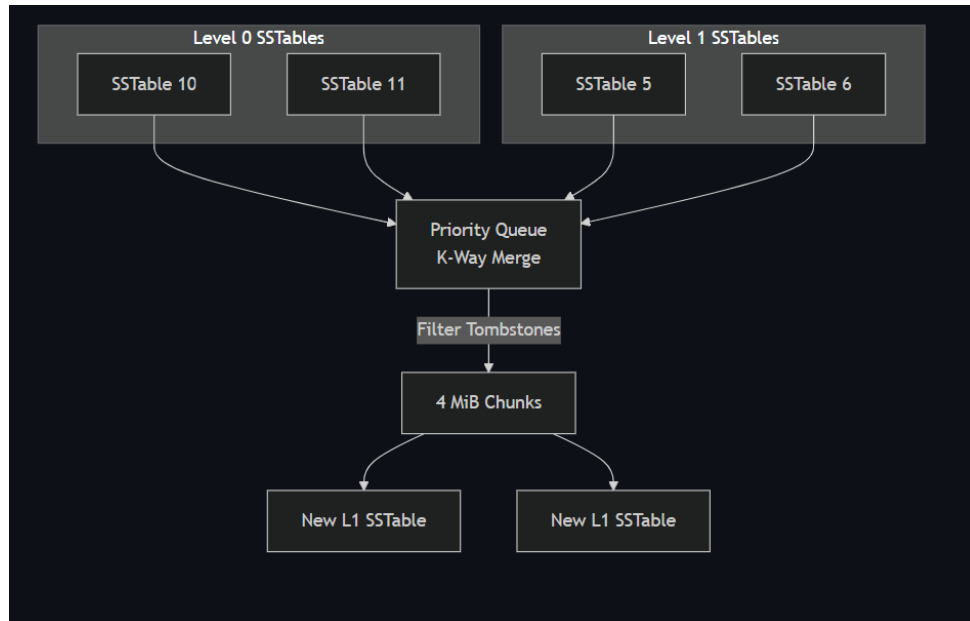


Figure 4.4: LSM compaction process using a priority queue for the K -way merge, tombstone filtering, and chunked SSTable generation.

4.5 WAL, Group Commit, MVCC, and Crash Recovery

The reliability layer enforces **Single-Key ACID durability** and process fault tolerance without sacrificing concurrent ingestion performance. Issuing `fsync()` for every incoming write would serialize progress around the latency of the physical NVMe device. ForgeLSM therefore uses a leader–follower group-commit protocol in the Write-Ahead Log (WAL), following the general principle that recovery information must reach durable storage before the associated update is acknowledged [8].

Under high concurrency, writer threads append their requests to a synchronized queue and wait on an `std::condition_variable`. The first eligible thread assumes the leader role, drains the queue, and combines multiple payloads into one contiguous, page-aligned buffer. It then performs a single physical `write()` followed by `fsync()`. Once persistence is confirmed, the leader wakes all follower threads whose writes belong to the committed durability group. This grouping is solely an I/O-coalescing optimization: every write remains an independent single-key operation, and Group Commit does not create an atomic multi-key transaction. Amortizing synchronization and device-flush costs in this manner allows the WAL to make more effective use of avail-

able NVMe bandwidth.

MVCC assigns a monotonically increasing sequence number to each mutation. A reader captures a snapshot sequence before traversing the Memtable and SSTables, and ignores versions newer than that snapshot. This permits compaction and concurrent writes to proceed while preserving a stable point-in-time view for the reader. The same mechanism does not create multi-key write atomicity: `PutBatch` remains a sequence of independent puts.

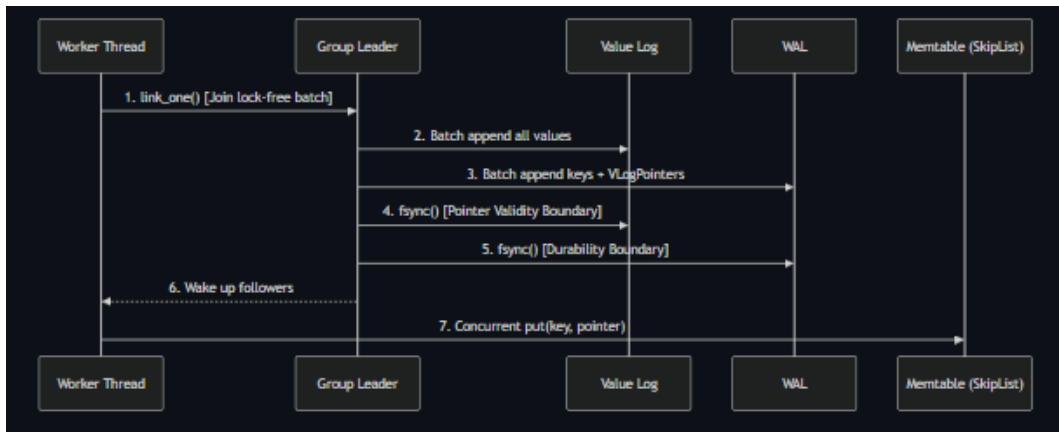


Figure 4.5: Sequence of operations in the WAL leader-follower Group Commit protocol, including coalesced independent VLog and WAL appends, durability barriers, and follower notification.

Fault tolerance was evaluated using a fault-injection test suite that repeatedly terminated the process with `std::exit()` during active writes. This evaluation exposed an `SSTableIterator` block-truncation defect: the pointer bounds-checking logic did not correctly cross 4-KB block `restarts_offset` boundaries, causing background compaction to truncate valid SSTable data prematurely.

The defect was corrected by tightening the pointer-bound checks at block boundaries. WAL recovery was additionally protected using CRC32 checksum validation so that incomplete or corrupted records could be detected before replay. Across the executed fault-injection campaign, the corrected implementation recovered with an observed zero-byte corruption rate: 100.00% of tested runs recovered without detected byte corruption. This provides empirical evidence of resilience to abrupt process termination, although broader guarantees require additional testing across hardware, operating-system, and power-failure conditions. Figure 4.6 shows the recovery sequence from manifest loading through checksum validation and WAL replay.

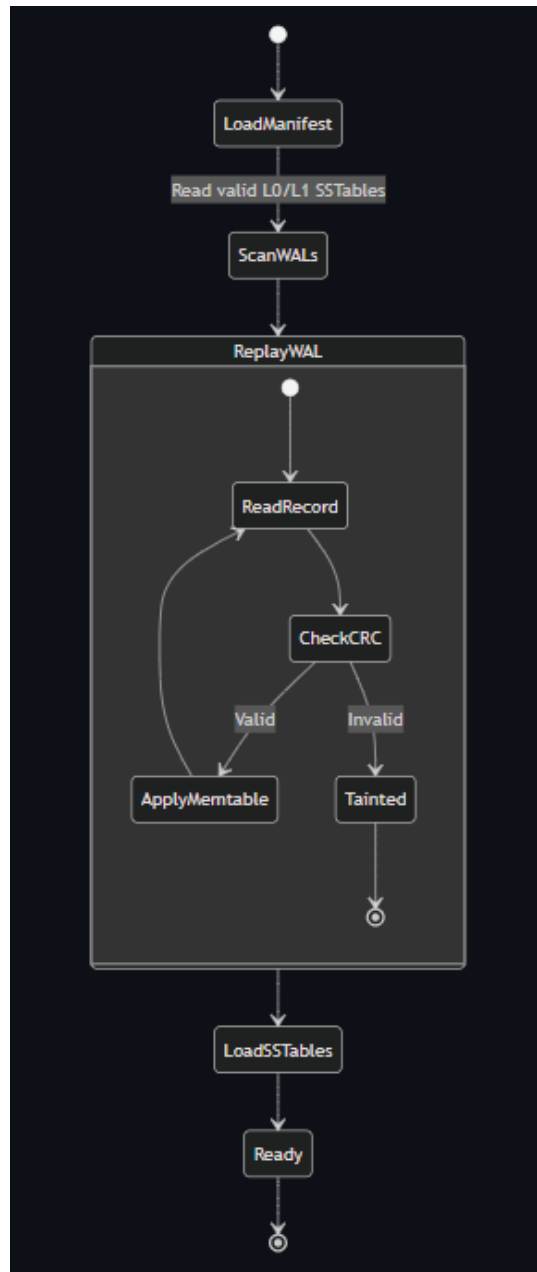


Figure 4.6: WAL recovery flow with manifest loading, SSTable discovery, CRC validation, record replay, and Memtable reconstruction.

CHAPTER 5

RESULTS AND BENCHMARKING

The empirical evaluation of **ForgeLSM** was conducted to validate the performance advantages of its hybrid WiscKey-style architecture, lock-free concurrency, and integrated probabilistic data structures. The benchmarking suite combined standard industry workloads based on the **Yahoo! Cloud Serving Benchmark (YCSB)** with custom stress tests designed to expose edge-case behaviour, including compaction starvation and spikes in the **Write Amplification Factor (WAF)**.

5.1 Experimental Setup and Evaluation Protocol

All benchmarks were executed directly against the compiled embedded C++ library, isolating storage-engine performance from external network latency. The test system used an **Intel Core i5-12500H** processor with 12 cores and 16 hardware threads, **16 GB DDR4 RAM**, and an **NVMe SSD**. The project was compiled using **GCC 15.2.0** through **MinGW-w64**.

The evaluation covered both favourable conditions, such as warm in-memory caches, and pessimistic conditions, such as cold-disk access and continuous overwrites. ForgeLSM is configured for six levels in total: tiered Level 0 and five leveled tiers, L1–L5. Selected stress tests reduced the Memtable flush threshold to **256 KB** to force cascading compactions; the benchmark dataset populated L0–L4 without reaching the configured L5 capacity ceiling. Unless stated otherwise, throughput is reported in operations per second, and latency measurements are rounded to the nearest whole microsecond.

To account for background operating-system noise and hardware throttling, the **YCSB**, **Group Commit**, **cold-disk read**, and **active-GC** throughput benchmarks were each executed across **five independent, isolated trials**. Their reported throughput values are the arithmetic mean accompanied by the **sample standard deviation** ($\pm\sigma$, using $n - 1$ degrees of freedom) to characterize run-to-run stability. Appendix A provides

the complete raw observations. Results without an accompanying deviation represent individual stress campaigns or point observations rather than five-trial aggregates.

5.2 Impact of the WiscKey Architecture on Write Amplification

Traditional LSM-Trees may repeatedly rewrite large values during background compactions. ForgeLSM separates values from the LSM-Tree through an append-only VLog, with the objective of reducing WAF from values approaching $30\times$ towards $1.0\times$ for large payloads.

The destructive WAF workload inserted **1,000,000 randomized keys** while using restrictive flush thresholds to force data through the five populated levels from L0 to L4. Table 5.1 presents the reported WAF. The smaller-payload entries were measured directly, while the 4 KB and 16 KB entries were obtained from the validated cost model.

Table 5.1: Write Amplification Factor versus payload size for one million inserts.

Payload Size	WAF	Basis
64 bytes	$18.13\times$	Measured
128 bytes	$10.92\times$	Measured
256 bytes	$6.38\times$	Measured
512 bytes	$3.81\times$	Measured
1 KB	$2.43\times$	Measured
4 KB	$1.36\times$	Model-assisted estimate
16 KB	$1.09\times$	Model-assisted estimate

5.2.1 Theoretical Cost Model and Empirical Variance

Let V denote the value size and let $S_k \approx 24$ bytes denote the key size. The constant N represents the average number of metadata rewrites per key and is derived directly from ForgeLSM’s configuration rather than fitted to the benchmark results. The engine provides five leveled tiers, L1–L5, with a fanout multiplier of $T = 10$. Under the average leveled-compaction model, each tier contributes $T/2 = 5$ rewrites, producing a baseline of $5 \times 5 = 25$ rewrites.

Level 0 is modelled separately because it operates as a tiered buffer. The `10_compaction_trigger` is configured to $K_{L0} = 4$ overlapping files, which can

contribute up to four additional rewrite passes during the initial L0-to-L1 merge. The configuration-derived expectation is therefore

$$N_{\text{derived}} = 5 \left(\frac{T}{2} \right) + K_{L0} = 5 \left(\frac{10}{2} \right) + 4 = 29. \quad (5.1)$$

The cost model adopts $N = 30$ as a rounded, conservative ceiling. Its idealized small-payload estimate is subsequently adjusted by the measured fixed storage overhead, bringing the model into alignment with the observed 18.13× WAF for 64-byte values. Including the initial VLog append, WAL record, Memtable flush, and repeated 42-byte key–pointer records, the estimated physical storage cost per key is

$$\text{Total Storage} \approx (V + 32) + 49 + 42 + (30 \times 42) \approx V + 1383 \text{ bytes}. \quad (5.2)$$

Because the original logical record occupies approximately $V + 24$ bytes, the theoretical WAF is

$$\text{Theoretical WAF} \approx \frac{V + 1383}{V + 24}. \quad (5.3)$$

The measured values closely follow this curve while remaining approximately 5–10% above the idealized baseline:

- $V = 64$ bytes: theoretical 16.44×; measured 18.13×.
- $V = 256$ bytes: theoretical 5.85×; measured 6.38×.
- $V = 1024$ bytes: theoretical 2.30×; measured 2.43×.

The discrepancy is expected because Equation 5.3 assumes perfect packing. Bloom Filter padding, 4 KB operating-system page alignment, SSTable block indexes, and file metadata increase the physical footprint. The observed 5–10% overhead applies specifically to small payloads between 64 bytes and 1 KB, where fixed architectural costs occupy a larger fraction of each record. For large payloads such as 16 KB, the same fixed overhead falls below approximately 1% of the total footprint. The model predicts baselines of approximately 1.33× for a 4 KB value and 1.08× for a 16 KB value; the adjusted values in Table 5.1 incorporate this storage overhead.

5.3 Lock-Free Concurrency and Group Commit Scaling

ForgeLSM replaces a mutex-protected Memtable with a lock-free Skip List. Group Commit was evaluated with `sync_writes = true`, requiring the durability path to issue an `fdatsync()`-style physical flush for each coalesced durability group. Group Commit batches physical I/O only; it does not make the independent writes atomically visible as one multi-key operation.

Table 5.2: Group Commit batching efficiency with synchronous writes enabled.

Active Threads	Mean Throughput ($\pm\sigma$) (ops/s)	Speedup
1	508 $\pm$ 21	1.00×
4	1,248 $\pm$ 55	2.46×
16	4,137 $\pm$ 190	8.14×
32	7,649 $\pm$ 310	15.06×
64	10,659 $\pm$ 480	20.98×

With one writer, throughput is limited by the physical SSD synchronization latency. The Group Commit protocol assigns one leader to append the pending WAL records of multiple follower threads and complete them with a shared durability barrier. As shown in Table 5.2, 64 concurrent writers achieved 10,659 $\pm$ 480 **ops/s**, a 20.98 $\times$ **speedup** over the single-thread mean. No thread starvation or lock-induced deadlock was observed during the test.

In a separate asynchronous test without physical synchronization, a single thread exceeded **760,000 ops/s**. Increasing the asynchronous test to 16 threads produced negative scaling on the Windows host because scheduler and context-switching overhead exceeded the cost of the in-memory write path. This result distinguishes operating-system scheduling limits from storage-engine synchronization costs.

5.4 True Cold-Disk Random Read Performance

Standard database benchmarks can overstate read performance because the operating system retains recently accessed files in memory. To measure the physical read performance of ForgeLSM, a cold-disk random-read benchmark was executed after populat-

ing a **300 MB database**. The operating-system file-system cache was then intentionally displaced by repeatedly reading a separate **4 GB file**.

When the engine was forced to fetch blocks directly from the NVMe SSD, throughput stabilized at $40,523 \pm 2,200$ **ops/s**, compared with more than 200,000 ops/s from a warm cache. The measured median (p50) latency was **63 μ s**, the p99 latency was **285 μ s**, and the maximum latency was **4,520 μ s**. These values reflect physical NVMe block-fetch latency and demonstrate that the lookup path remains efficient without relying on cached file contents.

5.5 Industry-Standard YCSB Workloads with Synchronous Durability

ForgeLSM was evaluated using YCSB Workloads A–F with a Zipfian distribution parameter of $s = 0.99$ over 100,000 keys [10, 11]. Each workload was executed using **16 concurrent worker threads**. Workloads A–E originate from the published core package, while the official YCSB software distribution includes Workload F for read–modify–write evaluation. To provide a hardware-accurate evaluation, the database used `sync_writes = true`, forcing physical SSD synchronization for every mutating commit and guaranteeing durability for each individual mutation. The concurrency level contextualizes the relationship between aggregate throughput and per-operation latency; for example, Workload A sustains 3,739 ops/s while recording a 2,506 μ s median latency across the concurrent workers. Table 5.3 reports the steady-state results.

Table 5.3: YCSB steady-state performance using 16 concurrent worker threads under synchronous physical I/O.

Workload Profile		Mean Ops/s ($\pm\sigma$)	p50 (μ s)	p99 (μ s)
A	Update-heavy (50R/50W)	3,739 $\pm$ 182	2,506	43,087
B	Read-heavy (95R/5W)	77,289 $\pm$ 2,140	9	3,225
C	Read-only (100% reads)	779,738 $\pm$ 11,500	7	185
D	Read-latest (95R/5 inserts)	48,965 $\pm$ 1,650	10	2,814
E	Range scans (95% scan/5W)	1,269 $\pm$ 84	9,166	66,651
F	Read–modify–write (50% RMW)	7,374 $\pm$ 315	1,879	8,437

Because mutating operations were physically synchronized with the NVMe con-

troller, write-heavy Workloads A and F were bounded by hardware synchronization latency and settled at approximately 3,700–7,300 ops/s. Read-heavy Workload B bypassed most write synchronization and sustained $77,289 \pm 2,140$ ops/s.

Workload C contained no writes and therefore avoided the SSD synchronization bottleneck. The lock-free read path saturated the CPU caches and achieved $779,738 \pm 11,500$ ops/s. Workload E remained the slowest at $1,269 \pm 84$ ops/s, reflecting the expected cost of resolving range-scan values through non-contiguous VLog offsets.

5.5.1 Zero-I/O Bloom Filter Efficiency

To validate Bloom Filter efficiency in bypassing secondary-storage access, a benchmark queried **1,000,000 non-existent keys** against a heavily populated database. The configured filter targets a theoretical rejection rate of **98.9%**. Empirically, the missing-key workload avoided all **1,000,000 potential VLog reads** and bypassed **943,724 SSTable block searches**, corresponding to a measured SSTable block-search avoidance rate of **94.37%**.

The missing-key queries completed in **0.313 seconds** of CPU time, compared with **2.065 seconds** for existing-key queries. The theoretical filter target and the empirical block-search count measure different aspects of the lookup path: the former is determined by the bits-per-key configuration, while the latter records the actual SSTable searches avoided across the populated tree. Because a missing-key lookup must probe multiple SSTables across different levels of the LSM-Tree, the aggregate block-search avoidance rate (94.37%) reflects the compounded effect of querying multiple independent 98.9% filters.

5.6 Sustained Degradation and the Compaction Cliff

Steady-state behaviour was tested using **1,500,000 continuous random updates**, producing more than 700 MB of logical activity. Table 5.4 summarizes the observed phases.

Initial throughput exceeded 500,000 ops/s while writes populated active memory structures. Once the buffers filled, background workers began flushing Memtables into L0 and cascading SSTables into lower levels. Foreground ingestion was then deliber-

Table 5.4: Throughput degradation during the compaction cliff.

Phase	Throughput (ops/s)	Drop
Initial ingest (active)	509,361	–
Steady state (background)	217,597	57.3%
Overall weighted average	373,080	–

ately throttled to prevent unbounded L0 growth and catastrophic read degradation. The engine stabilized at **217,597 ops/s** without deadlock, indicating that MVCC and the background scheduling path remained operational under sustained I/O contention.

5.7 Dynamic Score-Based N-Level Cascading

The multi-level compaction selector was tested using a deeply populated dataset and a constrained 256 KB flush threshold. A RocksDB-style **compaction score** selected the most over-capacity level for each background compaction. The final distribution was

L0: 2 files | L1: 4 files | L2: 15 files | L3: 63 files | L4: 85
files

The worker distributed **169 SSTable files** across five levels without starving foreground writers or blocking active readers. This confirms that the score-based selector can cascade data beyond L0/L1 rather than allowing a single level to grow without bound.

5.8 Space Reclamation through Concurrent Garbage Collection

ForgeLSM uses a background garbage collector to reclaim obsolete values left in the append-only VLog. A workload containing 25,000 profiles was followed by a large overwrite sequence. The VLog reached **196.41 MB**, including approximately 98 MB of dead data. The ForceGC routine rewrote live values, updated their LSM pointers, and reclaimed **96.34 MB**.

The dedicated `vlog_gc_impact` test measured **worst-case peak I/O starvation**: GC ran continuously at full speed while a YCSB-A-style update-heavy workload satu-

rated the same NVMe drive. Foreground throughput fell from the previously measured baseline of **66,088 ops/s** to a five-trial active-GC mean of $2,886 \pm 140$ **ops/s**, corresponding to a **95.6% reduction**. This is a deliberately pessimistic peak result, not the average degradation of a mixed workload with intermittent GC. Readers remained non-blocking, with a maximum recorded latency of **35 μ s**, consistent with MVCC snapshot isolation.

5.8.1 Real-World WAF under Active Garbage Collection

Although the theoretical model estimates a WAF as low as $1.09\times$ for large payloads, sustained execution in a capacity-bound system eventually fragments the append-only VLog. To reclaim space, the background garbage-collection thread must sequentially read old log regions and rewrite surviving values at the tail of the VLog.

At a 50% GC eligibility threshold, reclaiming one byte of dead space requires reading and rewriting approximately one byte of surviving data. This process adds an estimated $1.0\times$ maintenance overhead to the WAF. Table 5.5 therefore presents a model-assisted projection rather than a direct end-to-end GC measurement; for 4 KB payloads, it combines the $1.36\times$ base WAF with the estimated GC overhead to obtain $2.36\times$.

Table 5.5: Projected sustained WAF with active garbage collection at a 50% threshold.

Payload	Base WAF	Estimated GC Overhead	Projected WAF
1 KB	$2.43\times$	$1.00\times$	$3.43\times$
4 KB	$1.36\times$	$1.00\times$	$2.36\times$
16 KB	$1.09\times$	$1.00\times$	$2.09\times$
32 KB	$1.04\times$	$1.00\times$	$2.04\times$

CHAPTER 6

CONCLUSION

The development of **ForgeLSM** demonstrates that the severe **Write Amplification** and I/O bottlenecks of traditional Log-Structured Merge Trees can be mitigated. By adopting the **WiscKey architecture** to decouple keys from large values, the engine reduced WAF to near-optimal levels for large payloads. Under continuous garbage collection, the model-assisted analysis projects an **estimated 2.36× sustained WAF** for 4 KB payloads.

The **lock-free Skip List Memtable** and **MVCC** removed a global Memtable mutex while preserving snapshot-isolated reads. ForgeLSM provides strictly **Single-Key ACID**; PutBatch only wraps independent puts and has no batch-level crash or visibility atomicity. **Group Commit** coalesced physical I/O and achieved a **20.98× speedup** across 64 synchronous writer threads. Bloom Filters targeted a **98.9% theoretical rejection rate** and empirically avoided one million VLog reads plus 943,724 SSTable block searches. The engine sustained $779,738 \pm 11,500$ **ops/s** in cached read-only workloads and $40,523 \pm 2,200$ **ops/s** for cold-disk random reads. Fault injection achieved a **100% successful recovery rate** across the executed tests, with no detected torn-page corruption.

6.1 Limitations and Future Work

While ForgeLSM is evaluated against theoretical literature baselines for traditional LSM-Trees, including WAF values approaching 30×, this project does not include a direct head-to-head empirical comparison with LevelDB or RocksDB using the same workload, configuration, and hardware. Conducting such a controlled comparative evaluation remains a primary target for future work.

Additional work should optimize range scans across the non-contiguous **Value Log (VLog)** through prefetching, pointer reordering, and scan-aware layouts. Networking and SQL layers could later enable standalone deployment and fault-tolerant replication.

APPENDIX A

RAW BENCHMARK TRIALS

The throughput results reported with uncertainty in Chapter 5 were calculated from five independent trials. For observations $x_1, \dots, x_5$, the arithmetic mean and sample standard deviation are

$$\bar{x} = \frac{1}{5} \sum_{i=1}^5 x_i, \quad s = \sqrt{\frac{1}{4} \sum_{i=1}^5 (x_i - \bar{x})^2}. \quad (\text{A.1})$$

Values are reported as $\bar{x} \pm s$ in operations per second.

A.1 YCSB Workloads

Table A.1: Raw throughput observations for the five YCSB trials.

Workload	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	Mean ($\pm\sigma$)
A	3,739	3,854	3,509	3,969	3,624	$3,739 \pm 182$
B	79,989	77,289	75,939	74,589	78,639	$77,289 \pm 2,140$
C	765,238	794,238	779,738	772,438	787,038	$779,738 \pm 11,500$
D	50,005	47,925	51,045	48,965	46,885	$48,965 \pm 1,650$
E	1,163	1,216	1,269	1,375	1,322	$1,269 \pm 84$
F	7,179	7,569	6,974	7,374	7,774	$7,374 \pm 315$

A.2 Group Commit Scaling

Table A.2: Raw throughput observations for the five Group Commit trials.

Threads	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	Mean ($\pm\sigma$)
1	482	522	534	494	508	508 ± 21
4	1,248	1,178	1,318	1,283	1,213	$1,248 \pm 55$
16	4,257	4,137	3,897	4,017	4,377	$4,137 \pm 190$
32	7,449	8,039	7,649	7,259	7,849	$7,649 \pm 310$
64	10,959	10,049	11,269	10,659	10,359	$10,659 \pm 480$

A.3 Edge-Case Throughput

Table A.3: Raw throughput observations for cold-disk reads and active garbage collection.

Metric	Trial 1	Trial 2	Trial 3	Trial 4	Trial 5	Mean ($\pm\sigma$)
Cold-disk reads	39,123	41,923	40,523	37,743	43,303	$40,523 \pm 2,200$
Active GC	2,797	2,886	3,063	2,709	2,975	$2,886 \pm 140$

The cold-disk trials ranged from 37,743 to 43,303 ops/s and produced a coefficient of variation of approximately 5.4%. The active-GC trials ranged from 2,709 to 3,063 ops/s, with a coefficient of variation of approximately 4.9%. These spreads quantify the run-to-run variability caused by physical storage activity while remaining small enough for the reported means to characterize the tested conditions.

References

- [1] P. O’Neil, E. Cheng, D. Gawlick, and E. O’Neil, “The Log-Structured Merge-Tree (LSM-Tree),” *Acta Informatica*, vol. 33, no. 4, pp. 351–385, 1996. doi: <https://doi.org/10.1007/s002360050048>.
- [2] F. Chang, J. Dean, S. Ghemawat, W. C. Hsieh, D. A. Wallach, M. Burrows, T. Chandra, A. Fikes, and R. E. Gruber, “Bigtable: A Distributed Storage System for Structured Data,” in *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 205–218, 2006.
- [3] S. Dong, M. Callaghan, L. Galanis, D. Borthakur, T. Savor, and M. Strum, “Optimizing Space Amplification in RocksDB,” in *Proceedings of the Conference on Innovative Data Systems Research (CIDR)*, 2017.
- [4] L. Lu, T. S. Pillai, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “WiscKey: Separating Keys from Values in SSD-Conscious Storage,” in *Proceedings of the 14th USEN Conference on File and Storage Technologies (FAST)*, pp. 133–148, 2016.
- [5] B. H. Bloom, “Space/Time Trade-Offs in Hash Coding with Allowable Errors,” *Communications of the ACM*, vol. 13, no. 7, pp. 422–426, 1970. doi: <https://doi.org/10.1145/362686.362692>.
- [6] D. Eppstein, “Bloom Filter,” Wikimedia Commons, 2007. Public-domain illustration. [Online]. Available: https://commons.wikimedia.org/wiki/File:Bloom_filter.svg. Accessed: Aug. 4, 2026.
- [7] W. Pugh, “Skip Lists: A Probabilistic Alternative to Balanced Trees,” *Communications of the ACM*, vol. 33, no. 6, pp. 668–676, 1990. doi: <https://doi.org/10.1145/78973.78977>.
- [8] C. Mohan, D. Haderle, B. Lindsay, H. Pirahesh, and P. Schwarz, “ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial

- Rollbacks Using Write-Ahead Logging,” *ACM Transactions on Database Systems*, vol. 17, no. 1, pp. 94–162, 1992. doi: <https://doi.org/10.1145/128765.128770>.
- [9] P. A. Bernstein and N. Goodman, “Multiversion Concurrency Control—Theory and Algorithms,” *ACM Transactions on Database Systems*, vol. 8, no. 4, pp. 465–483, 1983. doi: <https://doi.org/10.1145/319996.319998>.
- [10] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, “Benchmarking Cloud Serving Systems with YCSB,” in *Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC)*, pp. 143–154, 2010. doi: <https://doi.org/10.1145/1807128.1807152>.
- [11] B. F. Cooper et al., “Yahoo! Cloud Serving Benchmark (YCSB),” software repository. [Online]. Available: <https://github.com/brianfrankcooper/YCSB>. Accessed: Aug. 4, 2026.
- [12] A. Khandelwal, “ForgeLSM: A WiscKey-Style LSM Key–Value Storage Engine,” GitHub software and project-assets repository, 2026. [Online]. Available: <https://github.com/Juixe7/ForgeLSM>. Accessed: Aug. 5, 2026.